

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Mendoza
Tecnicatura Universitaria en Programación

Food Store

Sistema de Gestión de Pedidos por Consola

Materia:	Programación 2
Ciclo Académico:	2do Semestre, 2026
Integrantes:	Matias Vargas Facundo Deseff Facundo Ramirez Santino Godoy
Fecha de Entrega:	22 de Junio de 2026

Índice General

1. Marco Teórico	3
1.1 Lenguaje de Programación Java y Versión 21	3
1.2 Persistencia Relacional y JDBC	3
1.3 Pool de Conexiones con HikariCP	3
1.4 Arquitectura de Software en Capas (N-Tier)	4
2. Decisiones Técnicas y Arquitectura	4
2.1 Estructura Completa del Árbol de Paquetes	4
2.2 Modelo de Datos y Relaciones SQL	5
2.3 Interfaz de Usuario y Ejecución Explicativa del Sistema	5
3. Dificultades Técnicas y Soluciones	6
3.1 Gestión Transaccional y Control de Consistencia	6
3.2 Implementación Concurrente de Pool de Conexiones	6
3.3 Manejo Centralizado de Excepciones de Negocio	6
4. Bibliografía y Enlaces del Proyecto	7

1. Marco Teórico

El desarrollo de sistemas de información robustos requiere de una base tecnológica sólida que combine un lenguaje de programación de alto rendimiento, un motor de base de datos eficiente y una arquitectura que minimice el acoplamiento entre componentes. El sistema **Food Store** se ha diseñado bajo estas premisas utilizando el ecosistema corporativo de Java.

1.1 Lenguaje de Programación Java y Versión 21

Se adoptó **Java 21** (versión con soporte de largo plazo o LTS). Esta versión provee optimizaciones críticas en el recolector de basura, mejoras notables en el rendimiento del compilador JIT y la consolidación de características sintácticas modernas como los *Record Patterns* y las mejoras en la coincidencia de patrones (Pattern Matching). El uso de características actualizadas asegura la vigencia técnica del software y una óptima gestión de la memoria.

1.2 Persistencia Relacional y JDBC

Para la capa de persistencia se optó por **MariaDB** como motor de base de datos relacional debido a su compatibilidad, velocidad y herencia de MySQL. En lugar de incorporar frameworks ORM pesados que abstraen por completo el lenguaje de consultas, se seleccionó **JDBC (Java Database Connectivity)**. JDBC permite un control granular absoluto sobre las sentencias ejecutadas, facilitando el uso de `PreparedStatement` para prevenir ataques de inyección SQL (SQLi) y garantizando tiempos de respuesta mínimos.

1.3 Pool de Conexiones con HikariCP

Abrir y cerrar una conexión física a la base de datos por cada transacción es una operación computacionalmente costosa que degrada el rendimiento de cualquier software. Para mitigar esto, se integró **HikariCP**, reconocido como el pool de conexiones JDBC más rápido y ligero disponible en el ecosistema Java. HikariCP mantiene un conjunto de conexiones pre-establecidas y activas, asignándolas de inmediato ante cada petición de la capa DAO y reutilizándolas eficientemente una vez finalizada la consulta.

1.4 Arquitectura de Software en Capas (N-Tier)

La consistencia estructural se logró mediante una arquitectura clásica dividida en capas lógicas bien definidas:

- **Capa de Vista (Main):** Responsable del despliegue visual de los menús en consola y de la captura ordenada de los parámetros ingresados por el operador.
- **Capa de Controladores (Controllers):** Actúa como intermediaria, abstrayendo la vista de la lógica dura. Captura los flujos de control y gestiona de manera unificada los bloques `try-catch` para errores del negocio.
- **Capa de Servicios (Services):** Reúne la totalidad de las reglas de negocio, validaciones de stock, restricciones de estados y orquestación de operaciones lógicas previas a la persistencia.
- **Capa de Acceso a Datos (DAO):** Aísla los comandos nativos SQL de las capas superiores mediante interfaces genéricas estructuradas bajo el patrón CRUD (Create, Read, Update, Delete).

2. Decisiones Técnicas y Arquitectura

2.1 Estructura Completa del Árbol de Paquetes

Con el fin de garantizar la modularidad, la transparencia y la facilidad de mantenimiento a largo plazo por múltiples desarrolladores, el código fuente se estructuró de manera semántica detallando cada uno de los archivos correspondientes a las capas operativas:

```

src/integrador/prog2/
├─ main.java                                # Punto de entrada y menú principal
interactivo
├─ config/
│   ├── DatabaseConnectionPool.java        # Pool de conexiones administrado con HikariCP
│   └─ schema.sql                          # Script de creación de tablas y datos de
prueba
├─ entities/                               # Entidades del modelo de dominio
│   ├── Base.java                          # Clase abstracta raíz (id, eliminado,
│   └─ createdAt)
│       ├── Calculable.java                # Interfaz contractual para el cálculo de
totales
│       ├── Producto.java
│       ├── Categoria.java
│       ├── Usuario.java
│       ├── Pedido.java
│       └─ DetallePedido.java
├─ enums/                                  # Enumeraciones del sistema
│   ├── Estado.java                        # PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
│   ├── FormaPago.java                    # TARJETA, TRANSFERENCIA, EFECTIVO
│   └─ Rol.java                            # ADMIN, USUARIO
├─ DAO/                                    # Capa de acceso a datos (Persistencia JDBC)
│   ├── GenericDAO.java                    # Interfaz genérica con operaciones CRUD
parametrizadas
│   ├── ProductoDAO.java
│   ├── CategoriaDAO.java
│   ├── UsuarioDAO.java
│   ├── PedidoDAO.java
│   └─ DetallePedidoDAO.java
├─ services/                               # Capa de lógica de negocio y validaciones
│   ├── ProductoService.java
│   ├── CategoriaService.java
│   ├── UsuarioService.java
│   └─ PedidoService.java
├─ controllers/                            # Capa de coordinación entre la Vista y los
Servicios
│   ├── ProductoController.java
│   ├── CategoriaController.java
│   ├── UsuarioController.java
│   └─ PedidoController.java
└─ exception/                              # Jerarquía de excepciones propias del dominio
    ├── ValidacionNegocioException.java
    ├── EntidadNoEncontradaException.java
    ├── StockInsuficienteException.java
    └─ PersistenciaException.java

```

2.2 Modelo de Datos y Relaciones SQL

El diseño de la base de datos relacional `pedidos_db` prioriza la normalización y la integridad referencial. A continuación se detallan las cardinalidades implementadas:

- **Categoría (1) a Producto (N):** Un producto obligatoriamente se suscribe a una categoría matriz mediante la clave foránea `id_categoria`. Una categoría puede alojar n productos.
- **Usuario (1) a Pedido (N):** Cada pedido se traza e imputa inequívocamente a un usuario del sistema (comprador o gestor) mediante la clave foránea `id_usuario`.
- **Pedido (1) a Detalle de Pedido (N) y Producto (1):** Se implementó una tabla intermedia de rotura `detalle_pedido` que asocia las entidades mediante claves foráneas compuestas. Permite asociar múltiples ítems a una misma orden de compra sin violar las formas normales de bases de datos.

Estrategia de Borrado Lógico: Ningún registro es eliminado físicamente de las tablas mediante comandos `DELETE`. Se incorporó una columna de control booleana denominada `eliminado` en la clase abstracta de herencia común `Base.java`. Esto asegura la trazabilidad histórica de auditoría técnica y financiera de toda la facturación comercial.

2.3 Interfaz de Usuario y Ejecución Explicativa del Sistema

Al ejecutarse el sistema en consola, se despliega una interfaz limpia y estructurada mediante menús interactivos numéricos. A continuación, se presenta el log de salida de consola real de un flujo completo y exitoso de registro de pedido, el cual incluye las trazas de inicialización del pool de conexiones:

```

=====
SISTEMA DE PEDIDOS (FOOD STORE)
=====
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

--- PEDIDOS ---
1. Listar
2. Listar por usuario
3. Crear
4. Actualizar estado
5. Actualizar forma de pago
6. Eliminar
0. Volver
Seleccione: 3
ID del usuario: 1
[main] INFO com.zaxxer.hikari.HikariDataSource - HikariPool-1 - Starting...
[main] INFO com.zaxxer.hikari.pool.HikariPool - HikariPool-1 - Added connection
org.mariadb.jdbc.Connection@20e2cbe0
[main] INFO com.zaxxer.hikari.HikariDataSource - HikariPool-1 - Start completed.

ID      Nombre                Precio    Stock    Categoria
-----
1      Hamburguesa Clásica        $4500.00   20      Hamburguesas
2      Hamburguesa Completa       $5500.00   15      Hamburguesas
3      PizzaMuzzarella           $6000.00   10      Pizzas
4      PizzaFugazzeta            $6800.00    8      Pizzas
5      CocaCola500ml             $1500.00   50      Bebidas
6      AguaMineral500ml          $1200.00   40      Bebidas
ID del producto (0 para finalizar): 4
Cantidad: 2
Producto agregado. Subtotal parcial: $13600.0

ID      Nombre                Precio    Stock    Categoria
-----
1      Hamburguesa Clásica        $4500.00   20      Hamburguesas
2      Hamburguesa Completa       $5500.00   15      Hamburguesas
3      Pizza Muzzarella           $6000.00   10      Pizzas
4      Pizza Fugazzeta            $6800.00    8      Pizzas
5      Coca Cola 500ml           $1500.00   50      Bebidas
6      Agua Mineral 500ml         $1200.00   40      Bebidas
ID del producto (0 para finalizar): 0
Total del pedido: $13600.0
Forma de pago (1-TARJETA, 2-TRANSFERENCIA, 3-EFECTIVO): 3

[PedidoController] Peticion recibida para registrar un pedido...
[Service] Iniciando validación profunda del pedido Nro: null...
Validando datos del usuario: Facundo...
El usuario 'Facundo Ramirez' es válido.
[Service] Pedido e integrantes verificados con éxito! Estado: CONFIRMADO
[Service] Enviando transaccion a la base de datos...
[Service] El pedido se impacto correctamente en la base de datos!

```

```
[PedidoController]Respuestaenviadaconexito.
```

```
---PEDIDOS---
```

1. Listar
2. Listarporusuario
3. Crear
4. Actualizareestado
5. Actualizarformadepago
6. Eliminar
- 0.

```
VolverSeleccione:
```


3. Dificultades Técnicas y Soluciones

Obstáculo Identificado	Análisis Técnico y Solución Aplicada
Pérdida de Consistencia en el Alta de Pedidos (Desajuste de Stock)	Al dar de alta un pedido complejo, si la inserción de un ítem de detalle fallaba o el producto se quedaba sin stock a mitad del proceso, los ítems previos se guardaban parcialmente, dejando la base de datos en un estado inválido. <i>Solución:</i> Se desactivó el modo <code>autoCommit</code> en la conexión JDBC compartida (<code>connection.setAutoCommit(false)</code>). Todo el proceso de validación e inserción fue encapsulado en un único bloque transaccional controlado. Si surge cualquier excepción, se invoca un <code>connection.rollback()</code> en el bloque <code>catch</code>, asegurando que el stock regrese a su estado original. Si todo es exitoso, se ejecuta <code>connection.commit()</code>.
Agotamiento de Conexiones por Fugas (Connection Leaks)	Durante pruebas de estrés por consola, la aplicación se congelaba de manera indefinida después de realizar múltiples búsquedas consecutivas. <i>Solución:</i> Tras inspeccionar los objetos JDBC, se detectó que los componentes <code>PreparedStatement</code> y <code>ResultSet</code> no liberaban los recursos asignados del pool de HikariCP. Se refactorizó la totalidad de los métodos en la capa DAO aplicando el patrón estructural try-with-resources de Java. Esto garantiza el cierre automático implícito de conexiones al salir del bloque de ejecución.
Mezcla de Errores de Base de Datos con Errores de Negocio	Originalmente, los errores de sintaxis SQL o restricciones lógicas (ej: falta de stock) lanzaban excepciones genéricas <code>SQLException</code> que exponían detalles internos al operador de consola. <i>Solución:</i> Se diseñó una jerarquía especializada de excepciones heredadas de <code>RuntimeException</code>. La capa <code>Service</code> intercepta problemas lógicos y lanza errores específicos (<code>StockInsuficienteException</code>, etc.). La capa <code>Controller</code> centraliza estas capturas, mostrando mensajes claros y amigables por consola sin comprometer información interna del servidor.

4. Bibliografía y Enlaces del Proyecto

Para la realización y fundamentación técnica de este trabajo integrador, se consultaron las siguientes fuentes oficiales y especificaciones de la industria:

- **Oracle Java SE 21 Documentation:** Enlaces y especificaciones oficiales del lenguaje y la API JDBC.

<https://docs.oracle.com/en/java/javase/21/>

- **MariaDB Server Documentation:** Manual de referencia oficial para la optimización de sentencias DDL y relaciones foráneas.

<https://mariadb.com/kb/en/documentation/>

- **Bases Teóricas sacadas del Aula virtual:** Guías y referencias sobre los temas puestos en práctica durante la elaboración del proyecto.

<https://campusvirtual.frm.utn.edu.ar/login/index.php>

Links del Repositorio

El código fuente completo del proyecto, los scripts automáticos de la base de datos y el archivo de configuración POM de Maven se encuentran disponibles en el repositorio controlado de Git corporativo:

Repositorio:https://github.com/maaaattttiiii/TRABAJO_INTEGRADOR_PROGRAMACION2_MatiasVargas_FacundoDeseff_FacundoRamirez_SantinoGodoy.git